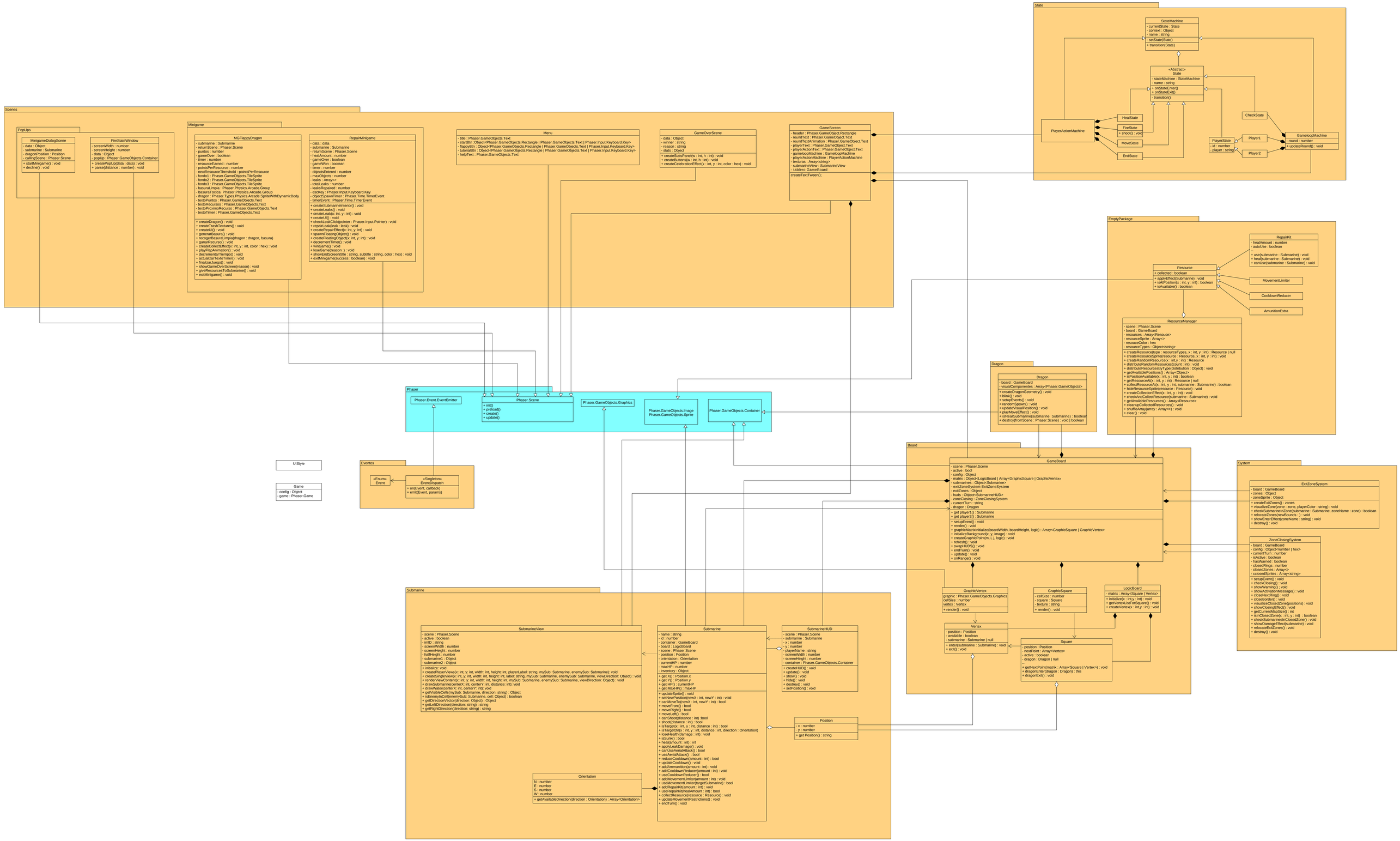
State
StateMachine
- currentState : State
- context : Object
- name : string
- setState(State)
+ transition(State)
«Abstract»
State
- stateMachine : StateMachine
- name : string
+ onStateEnter()
+ onStateExit()
- transition()
PlayerActionMachine
HealState
FireState
+ shoot() : void
MoveState
EndState
CheckState
PlayerState
- id : number
- player : string
Player1
Player2
GameloopMachine
- round : number
+ updateRound() : void
Scenes
PopUps
MinigameDialogScene
- data : Object
- submarine : Submarine
- dragonPosition : Position
- callingScene : Phaser.Scene
+ startMinigame() : void
+ decline() : void
FireStateWindow
- screenWidth : number
- screenHeight : number
- data : Object
- popUp : Phaser.GameObjects.Container
+ createPopUp(data : data) : void
+ parse(distance : number) : void
Minigame
MGFlappyDragon
- submarine : Submarine
- returnScene : Phaser.Scene
- puntos : number
- gameOver : boolean
- timer : number
- resourceEarned : number
- pointsPerResource : number
- nextResourceThreshold : pointsPerResource
- fondo1 : Phaser.GameObjects.TileSprite
- fondo2 : Phaser.GameObjects.TileSprite
- fondo3 : Phaser.GameObjects.TileSprite
- basuraLimpia : Phaser.Physics.Arcade.Group
- basuraToxica : Phaser.Physics.Arcade.Group
- dragon : Phaser.Types.Physics.Arcade.SpriteWithDynamicBody
- textoPuntos : Phaser.GameObjects.Text
- textoRecursos : Phaser.GameObjects.Text
- textoProximoRecurso : Phaser.GameObjects.Text
- textoTimer : Phaser.GameObjects.Text
+ createDragon() : void
+ createTrashTextures() : void
+ createUI() : void
+ generarBasura() : void
+ recogerBasuraLimpia(dragon : dragon, basura)
+ ganarRecurso() : void
+ createCollectEffect(x: int, y : int, color : hex) : void
+ playFlapAnimation() : void
+ decrementarTiempo() : void
+ actualizarTextoTimer() : void
+ finalizarJuego() : void
+ showGameOverScreen(reason) : void
+ giveResourcesToSubmarine() : void
+ exitMinigame() : void
RepairMinigame
- data : data
- submarine : Submarine
- returnScene : Phaser.Scene
- healAmount : number
- gameOver : boolean
- gameWon : boolean
- timer : number
- objectsEntered : number
- maxObjects : number
- leaks : Array<>
- totalLeaks : number
- leaksRepaired : number
- escKey : Phaser.Input.Keyboard.Key
- objectSpawnTimer : Phaser.Time.TimerEvent
- timerEvent : Phaser.Time.TimerEvent
+ createSubmarineInterior() : void
+ createLeaks() : void
+ createLeak(x: int, y : int) : void
+ createUI() : void
+ checkLeakClick(pointer : Phaser.Input.Pointer) : void
+ repairLeak(leak : leak) : void
+ createRepairEffect(x: int, y: int) : void
+ spawnFloatingObject() : void
+ createFloatingObject(x: int, y: int) : void
+ decrementTimer() : void
+ winGame() : void
+ loseGame(reason) : void
+ showEndScreen(title : string, subtitle : string, color : hex) : void
+ exitMinigame(success : boolean) : void
Menu
- title : Phaser.GameObjects.Text
- startBtn : Object<Phaser.GameObjects.Rectangle | Phaser.GameObjects.Text | Phaser.Input.Keyboard.Key>
- flappyBtn : Object<Phaser.GameObjects.Rectangle | Phaser.GameObjects.Text | Phaser.Input.Keyboard.Key>
- tutorialBtn : Object<Phaser.GameObjects.Rectangle | Phaser.GameObjects.Text | Phaser.Input.Keyboard.Key>
- helpText : Phaser.GameObjects.Text
GameOverScene
- data : Object
- winner : string
- reason : string
- stats : Object
+ createStatsPanel(w : int, h : int) : void
+ createButtons(w : int, h : int) : void
+ createCelebrationEffect(x : int, y : int, color : hex) : void
GameScreen
- header : Phaser.GameObject.Rectangle
- roundText : Phaser.GameObject.Text
- roundTextAnimation : Phaser.GameObject.Text
- playerText : Phaser.GameObject.Text
- playerActionText : Phaser.GameObject.Text
- gameloopMachine : GameloopMachine
- playerActionMachine : PlayerActionMachine
- texturas : Array<string>
- submarineView : SubmarineView
- tablero GameBoard
createTextTween();
EmptyPackage
RepairKit
- healAmount : number
- autoUse : boolean
+ use(submarine : Submarine) : void
+ heal(submarine : Submarine) : void
+ canUse(submarine : Submarine) : void
Resource
+ collected : boolean
+ applyEffect(Submarine) : void
+ isAtPosition(x : int, y : int) : boolean
+ isAvailable() : boolean
MovementLimiter
CooldownReducer
AmuntitionExtra
ResourceManager
- scene : Phaser.Scene
- board : GameBoard
- resources : Array<Resouce>
- resourceSprite : Array<>
- resoucecolor : hex
- resourceTypes : Object<string>
+ createResource(type : resourceTypes, x : int, y : int) : Resource | null
+ createResourceSprite(resource : Resource, x : int, y : int) : void
+ createRandomResource(x : int,y : int) : Resource
+ distributeRandomResources(count : int) : void
+ distributeResourcesByType(distribution : Object) : void
+ getAvailablePositions() : Array<Object>
+ isPositionAvailable(x : int, y : int) : boolean
+ getResourceAt(x : int, y : int) : Resource | null
+ collectResourceAt(x : int, y : int, submarine : Submarine) : boolean
+ hideResourceSprite(resource : Resource) : void
+ createCollectionEffect(x : int, y : int) : void
+ checkAndCollectResource(submarine : Submarine) : void
+ getAvailableResources() : Array<Resource>
+ cleanupCollectedResources() : void
+ shuffleArray(array : Array<>) : void
+ clear() : void
Dragon
Dragon
- board : GameBoard
- visualComponentes : Array<Phaser.GameObjects>
+ createDragonGeometry() : void
+ blink() : void
+ setupEvents() : void
+ randomSpawn() : void
+ updateVisualPosition() : void
+ playMoveEffect() : void
+ isNearSubmarine(submarine Submarine) : boolear
+ destroy(fromScene : Phaser.Scene) : void | boolean
Phaser
Phaser.Event.EventEmitter
Phaser.Scene
+ init()
+ preload()
+ create()
+ update()
Phaser.GameObjects.Graphics
Phaser.GameObjects.Image
Phaser.GameObjects.Sprite
Phaser.GameObjects.Container
UIStyle
Game
- config : Object
- game : Phaser.Game
Eventos
«Enum»
Event
«Singleton»
EventDispatch
+ on(Event, callback)
+ emit(Event, params)
Board
GameBoard
- scene : Phaser.Scene
- active : bool
- config : Object
- matrix : Object<LogicBoard | Array<GraphicSquare | GraphicVertex»
- submarines : Object<Submarine>
- exitZoneSystem ExitZoneSystem
- exitZones : Object
- huds : Object<SubmarineHUD>
- zoneClosing : ZoneClosingSystem
- currentTurn : string
- dragon : Dragon
+ get player1() : Submarine
+ get player2() : Submarine
+ setupEvent() : void
+ render() : void
+ graphicMatrixInitialize(boardWidth, boardHeight, logic) : Array<GraphicSquare | GraphicVertex>
+ initializeBackground(x, y, image) : void
+ createGraphicPoint(m, i, j, logic) : void
+ refresh() : void
+ swapHUDS() : void
+ endTurn() : void
+ update() : void
+ onRange() : void
GraphicVertex
graphic : Phaser.GameObjects.Graphics
cellSize : number
vertex : Vertex
+ render() : void
GraphicSquare
- cellSize : number
- square : Square
- texture : string
+ render() : void
LogicBoard
- matrix : Array<Square | Vertex>
+ initialize(x : int,y : int) : void
+ getVertexListForSquare() : void
+ createVertex(x : int,y : int) : void
Vertex
- position : Position
- available : boolean
- submarine : Submarine [null
+ enter(submarine : Submarine) : void
+ exit() : void
Square
- position : Position
- nextPoint : Array<Vertex>
- active : boolean
- dragon : Dragon [null
+ getNextPoint(matrix : Array<Square | Vertex>) : void
+ dragonEnter(dragon : Dragon) : this
+ dragonExit() : void
System
ExitZoneSystem
- board : GameBoard
- zones : Object
- zoneSprite : Object
+ createExitZones() : zones
+ visualizeZone(zone : zone, playerColor : string) : void
+ checkSubmarineInZone(submarine : Submarine, zoneName : zone) : boolean
+ relocateZones(newBounds) : void
+ showEnterEffect(zoneName : string) : void
+ destroy() : void
ZoneClosingSystem
- board : GameBoard
- config : Object<number | hex>
- currentTurn : number
- isActive : boolean
- hasWarned : boolean
- closedRings : number
- closedZones : Array<>
- cclosedSprites : Array<string>
+ setupEvent() : void
+ checkClosing() : void
+ showWarning() : void
+ showActivationMessage() : void
+ closeNextRing() : void
+ closeBorder() : void
+ visualizeClosedZone(positions) : void
+ showClosingEffect() : void
+ getCurrentMapSize() : int
+ isInClosedZone(x : int, y : int) : boolean
+ checkSubmarinesInClosedZone() : void
+ showDamageEffect(submarine) : void
+ relocateExitZones() : void
+ destroy() : void
Submarine
SubmarineView
- scene : Phaser.Scene
- active : boolean
- imID : string
- screenWidth : number
- screenHeight : number
- halfHeight : number
- submarine1 : Object
- submarine2 : Object
+ initialize: void
+ createPlayerViews(x: int, y: int, width: int, height: int, playerLabel: string, mySub: Submarine, enemySub: Submarine): void
+ createSingleView(x: int, y: int, width: int, height: int, label: string, mySub: Submarine, enemySub: Submarine, viewDirection: Object) : void
+ renderViewContent(x: int, y: int, width: int, height: int, mySub: Submarine, enemySub: Submarine, viewDirection: Object) : void
+ drawSubmarine(centerX: int, centerY: int, distance: int): void
+ drawWater(centerX: int, centerY: int): void
+ getVisibleCells(mySub: Submarine, direction: string) : Object
+ isEnemyInCell(enemySub: Submarine, cell: Object) : boolean
+ getDirectionVector(direction: Object) : Object
+ getLeftDirection(direction: string) : string
+ getRightDirection(direction: string) : string
Submarine
- name : string
- id : number
- container : GameBoard
- board : LogicBoard
- scene : Phaser.Scene
- position : Position
- orientation : Orientation
- currentHP : number
- maxHP : number
- inventory : Object
+ get X() : Position.x
+ get Y() : Position.y
+ get HP() : currentHP
+ get MaxHP() : maxHP
+ updateSprite() : void
+ setNewPosition(newX : int, newY : int) : void
+ canMoveTo(newX : int, newY : int) : bool
+ moveFront() : bool
+ moveRight() : bool
+ moveLeft() : bool
+ canShoot(distance : int) :bool
+ shoot(distance : int) : bool
+ isTarget(x : int, y : int, distance : int) : bool
+ isTargetDe(x : int, y : int, distance : int, direction : Orientation)
+ loseHealth(damage : int) : void
+ isSunk() : bool
+ heal(amount : int) : int
+ applyLeakDamage() : void
+ canUseAerialAttack() : bool
+ useAerialAttack() : bool
+ reduceCooldown(amount : int) : bool
+ updateCooldown() : void
+ addAmmunition(amount : int) : void
+ addCooldownReducer(amount : int) : void
+ useCooldownReducer() : bool
+ addMovementLimiter(amount : int) : void
+ useMovementLimiter(targetSubmarine) : bool
+ addRepairKit(amount : int) : void
+ useRepairKit(healAmount : int) : bool
+ collectResource(resource : Resource) : void
+ updateMovementRestrictions() : void
+ endTurn() : void
SubmarineHUD
- scene : Phaser.Scene
- submarine : Submarine
- x : number
- y : number
- playerName : string
- screenWidth : number
- screenHeight : number
- container : Phaser.GameObjects.Container
+ createHUD() : void
+ update() : void
+ show() : void
+ hide() : void
+ destroy() : void
+ setPosition() : void
Position
- x : number
- y : number
+ get Position() : string
Orientation
N : number
E : number
S : number
W : number
+ getAvailableDirection(direction : Orientation) : Array<Orientation>